Installing Software for Neuropixels analysis

This document describes setup for basic Neuropixels analysis, duplicating the environment that will be available on the course laptops. For the course data (specifically, the demonstration datasets and the data you'll be acquiring) a laptop with >16G RAM and a GPU with a Passmark rating > 10000 will be sufficient. If you are setting up an analysis workstation, there are guidelines for computer shopping here (https://github.com/billkarsh/SpikeGLX/blob/master/Markdown/SystemRequirements_PXI.md#spike-sorting-computer)

The goal of this exercise is for you to set up and run spike sorting, specifically on the test dataset. The first virtual session will cover the mechanics of Neuropixels data analysis. The 2nd remote session will cover working with the output of sorter data. Ideally, you will be able to run the sorter on the sample dataset, and be ready to follow along with interpreting the results.

If you are working on a system that already has sorting software installed, but want to learn how to do it yourself – go for it! Make sure that any virtual environments you create have new names.

These instructions are for the Windows OS. Most of the same instructions will work in Linux, but the testing was run in Windows.

You should try to run this setup before the first virtual session. Make notes about any questions you have (why are we doing that?) and any problems you encounter. You can post questions on the Slack channel. We'll address as many issues as we can there, and tackle remaining problems during the session.

Important Note – If you don't have a computer with an appropriate GPU

You can still follow all of these instructions except for installing Kilosort, and download the results from this google drive: https://drive.google.com/file/d/1cnxe4GcTI4recrZA3vY52SwfwOry5fU-/view

Chrome (if not already present):

NOTE: We encourage using Chrome for the best compatibility with Jupyter lab.

https://support.google.com/chrome/answer/95346?hl=en&co=GENIE.Platform%3DDesktop

Miniforge:

NOTE: If you already have Anaconda or Miniconda installed, feel free to use that instead. We encourage using Miniforge because there are no licensing fees for packages installed from its channels.

GitHub - conda-forge/miniforge: A conda-forge distribution.
Installer for windows included in 'software_dowloads' folder
Default installation location
Do not add to environment path.
Register python as default for the system
Remove package cache

After installation, create an icon on the desktop

Git bash:

NOTE: We'll be accessing github through VSCode. If you already have any form of git installed, VSCode should work with that installation, and you should skip this step.

https://git-scm.com/install/windows

Installer included in software downloads, copy to local computer to avoid permissions issues. If a dialog is missing from this list, use the defaults.

Accept license
Default location
Default components
Default vim editor
Switch from default to main for initial branch
Default git from command line and 3rd party
Default bundled SSH exe
Default Windows Secure Channel Library
Default line endings (windows checkout, unix check in)
Default MinTTY for terminal
Default git pull
Default credential manager
Default extra config options

VSCode:

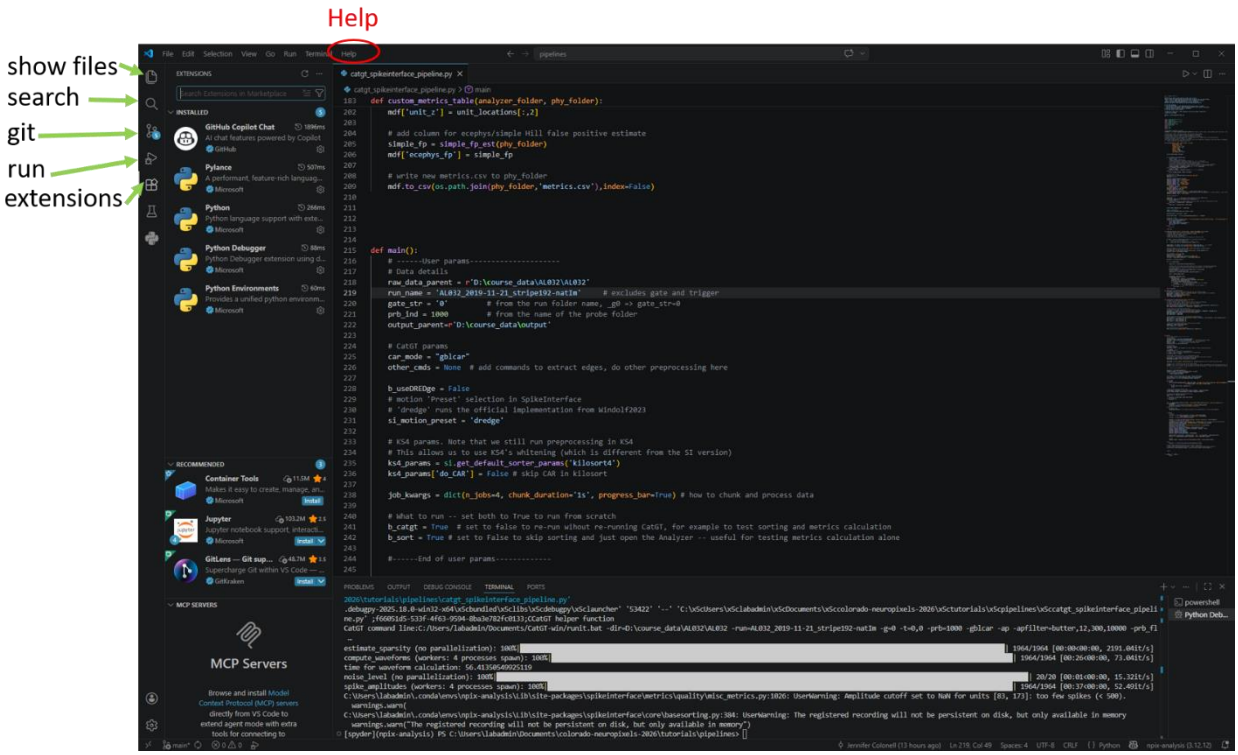
NOTE: The sorting pipeline demos for the course will be run with VSCode…but it can be run with any python IDE.

https://code.visualstudio.com/download

accept license agreement
default location OK

In "Additional Tasks" – if you want to minimize the impact of VS Code on your system – for example, if you use a different IDE and want it to be the default everywhere, uncheck all. Otherwise keep defaults.

A few useful features in the VSCode interface are shown in the screenshot below. There's a great cheat sheet for VS code here: https://www.learnbycheating.com/subjects/vscode/cheatsheet



After opening VSCode, click on extensions, and install:

Python
Pylance
Python Debugger
Python Environments
Jupyter (this is a package of 4 extensions, install the group
Jupyter Cell Tags
Jupyter Keymap
Jupyter Notebook Renderer
Jupyter Slide Show

Clone the course repo:

Navigate to: https://github.com/jcouto/UCLA-neuropixels-2026
Click on the green <>Code button and copy the address for the http address for the repo
In VS Code, Help->Show all commands or ctrl-shift-P.
In the popup search for/select Git:Clone
Paste in the address.
VSCode will bring up a file dialog to choose the destination.
Select 'Open the repository' – this will make it a new VS code workspace.

To install environments:

NOTE: These will make new, unique environments, and won't step on any other software installations, as long as the environment names are different.

npix-test environment

From miniforge (or anaconda/miniconda) terminal:

```
conda create -n npix-test python=3.12 pandas natsort ipympl ipywidgets
jupyterlab_widgets boto3 pyqt pyqtgraph scikit-learn scipy jupyterlab
zarr
```

You'll have to hit 'Y' for the installation to proceed. When that's complete, activate the environment.

```
conda activate npix-test
pip install kilosort[gui]
pip uninstsll torch
```

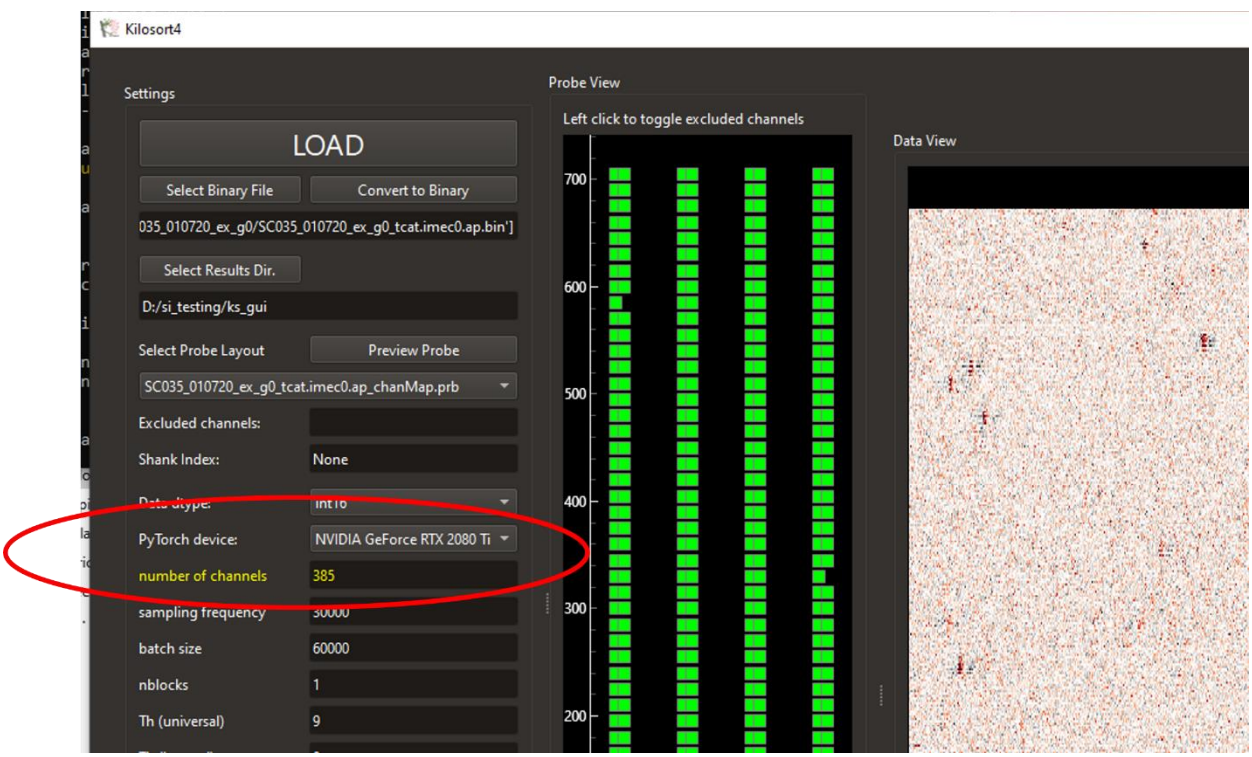
The last command removes the cpu version of torch that comes with the kilosort install. Next install pytorch with cuda:

```
pip3 install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu124
```

Test KS4 by running:

```
python -m kilosort
```

The most common problem is that pytorch won't be able to "see" the GPU. The KS4 interface shows the active in the PyTorch device field. If it is not a GPU card, torch will need to be reinstalled.



This can occur due to incompatibility of the CUDA toolkit with the GPU or drivers. The command above installs CUDA toolkit version 12.4, which should work with pretty much any GPU currently available, but if you're working with an older system a good choice is toolkit version 11.8. To switch to that version:

```
pip uninstsll torch
pip3 install torch --index-url https://download.pytorch.org/whl/cu118
```

There's more information about debugging a Kilosort install in the github repo: https://github.com/MouseLand/Kilosort

Finally, install spikeinterface:

```
pip install spikeinterface
```

phy2 environment

Phy is a standard visualization environment for looking at sort results. To install, first go back to the base environment:

```
conda deactivate
```

Use the instructions from https://github.com/cortex-lab/phy, but change the environment name (this will prevent disrupting any previous phy installation):

```
conda create -n phy2-test -y python=3.11 cython dask h5py joblib
matplotlib numpy pillow pip pyopengl pyqt pyqtwebengine pytest python
qtconsole requests responses scikit-learn scipy traitlets
```

Activate the environment and install the latest phy:

```
conda activate phy2-test
pip install git+https://github.com/cortex-lab/phy.git
```

SpikeGLX and CatGT installation (Windows)

The test data was acquired with SpikeGLX, and we'll use CatGT for preprocessing. Make a new folder in a location where you have write access. For windows, download the standard SpikeGLX latest release and the windows version of CatGT, and simply unzip them in your new folder.

https://billkarsh.github.io/SpikeGLX/App/Release_v20251218-api4.zip
https://billkarsh.github.io/SpikeGLX/Support/CatGTWinApp.zip

SpikeGLX and CatGT installation (Linux)

https://billkarsh.github.io/SpikeGLX/Support/SpikeGLX_Linux_offline.zip

After unzipping, you may need to run:

```
chmod +x SpikeGLX-x86_64.AppImage
```

For CatGT, get the Linux version of the app:

https://billkarsh.github.io/SpikeGLX/Support/CatGTLnxApp.zip

To install:

1. Copy CatGT-linux to your machine, cd into folder.
2. If needed, > chmod +x install.sh
3. > ./install.sh

Download the test data set

Create a folder for the raw data and output, on a drive with at least 100G of spare space. The folder name should not contain spaces.

The data download and later data visualization examples are presented in Jupyter lab notebooks. These can be opened in VSCode (more reliable) but should also work inside a browser (if you are using some other python IDE).

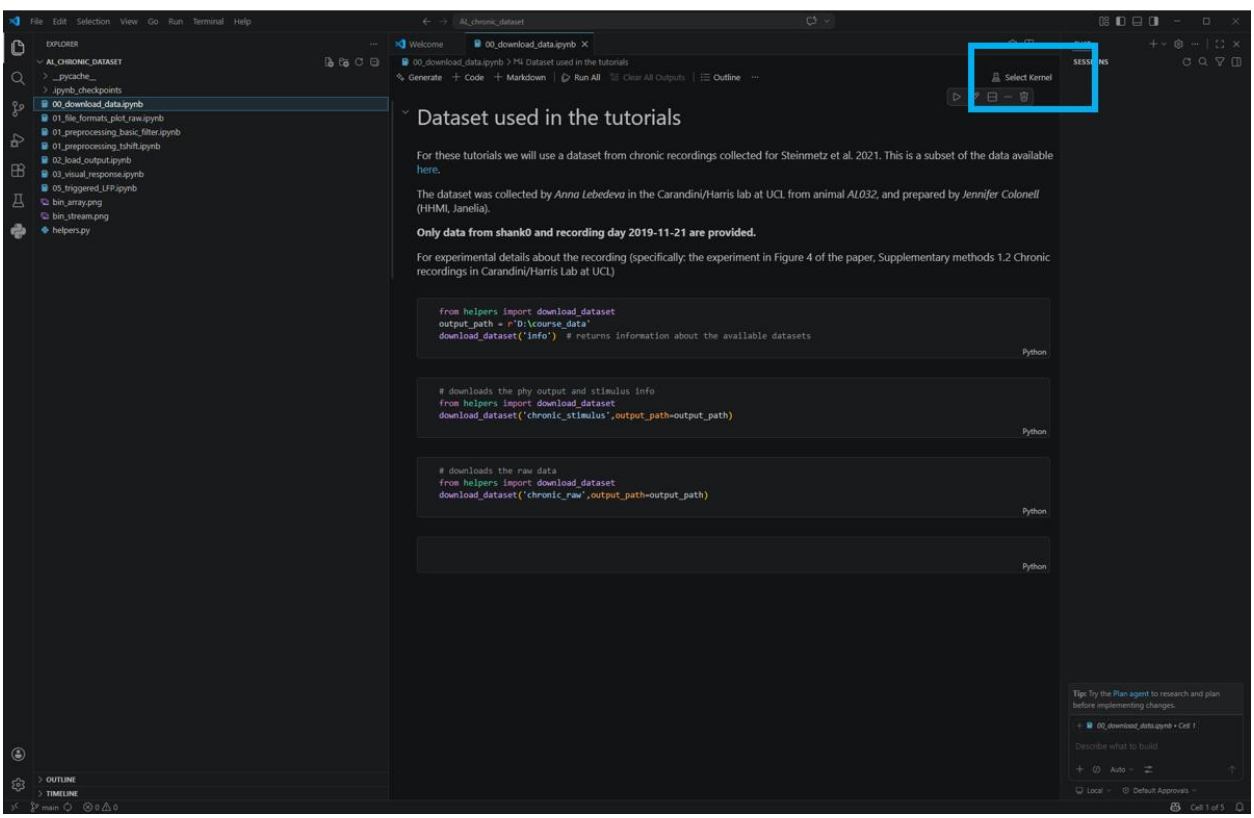
Opening the notebook in VSCode

File->Open folder to open:

colorado-neuropixels-2026/tutorials/AL_chronic_datasets

Click on 00_download_data.ipynb to open the notebook. In the upper right, click on “Select kernel” (see screenshot) and select the npix-test environment. Skip down to “Running the notebook” to proceed.

Select kernel



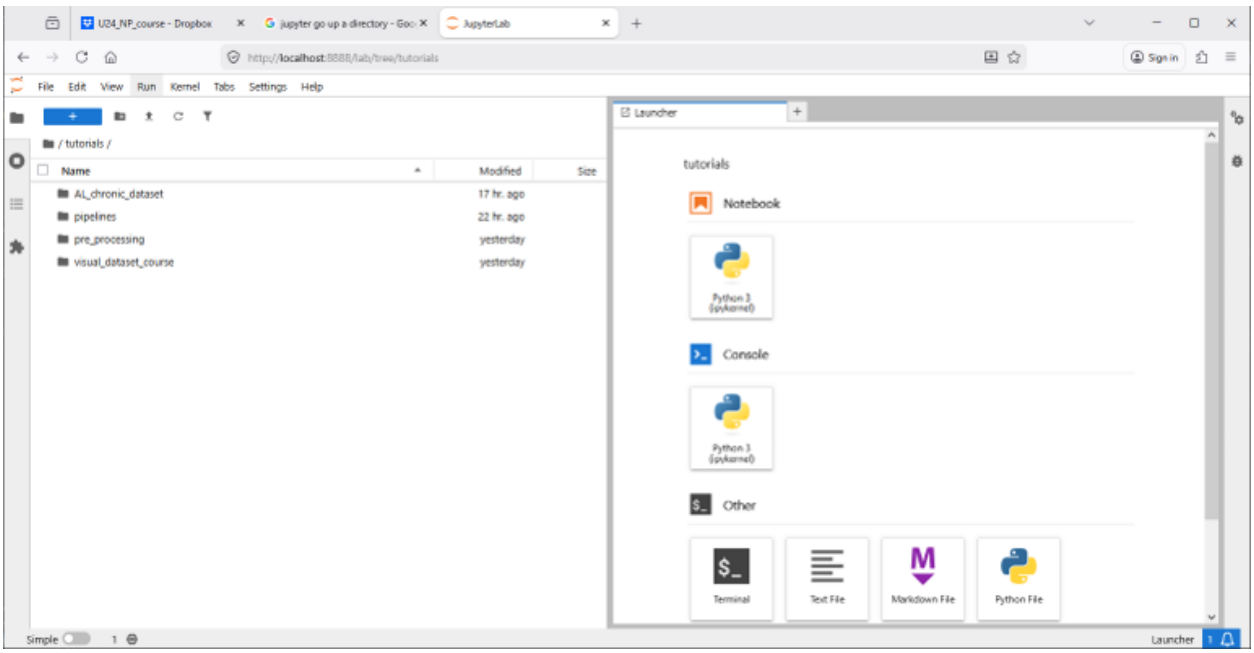
Opening the notebook in a browser

Open a miniforge (conda) terminal window and activate npix-test. Then navigate to the AL_chronic_dataset folder in the course repo, and start up jupyter lab:

```
conda activate npix-test
cd C:\Users\colonellj\Documents\colorado-neuropixels-2026
jupyter lab
```

jupyter lab will open in your default browser. If you run into any trouble with particular functions (especially interactive plots), it will probably work better in Chrome. The easiest way to force jupyter lab to open in Chrome is to set it to the default temporarily.

You'll get a screen like this, opened in the browser:



Double-click to open the folder AL_chronic_dataset, and then double click on 00_download_data.ipynb to open a notebook to download the test data.

Running the notbook

Each block of code (cell) is run by hitting shift + return (you'll have to select the first one).

Before you run, change the variable output_path in the 2nd block to the output folder created above. If using windows paths, use the prefix r to avoid interpretation of the backslashes, e.g.:

```
output_path=r'D:\course_data'
```

Run each block of the code to download the raw data, stimulus times, and sample output to your computer. The raw data (in subfolder AL032) is in a zip archive, which must be extracted before running the pipeline.

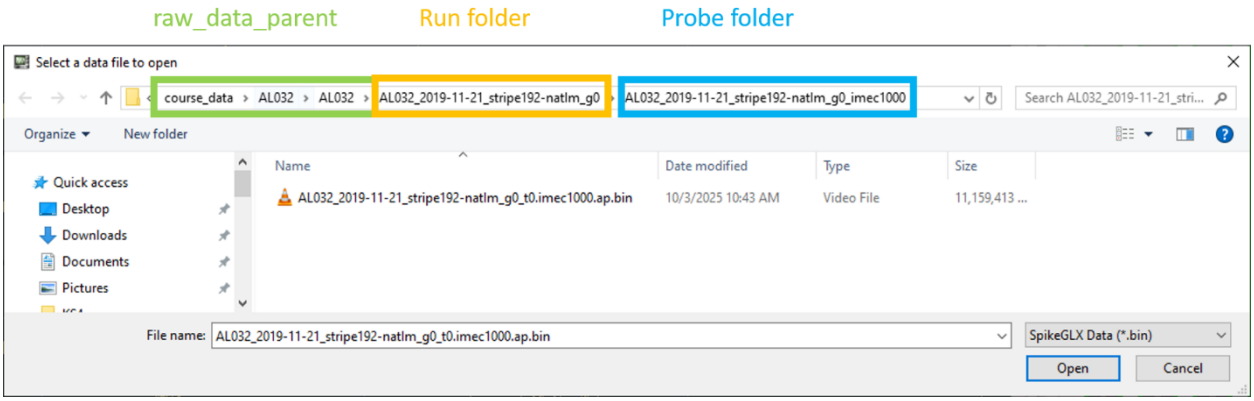
Open the raw data in the SpikeGLX viewer

This data was collected using SpikeGLX. SpikeGLX data is in run folders, which contain a probe folder for each probe in the recording:

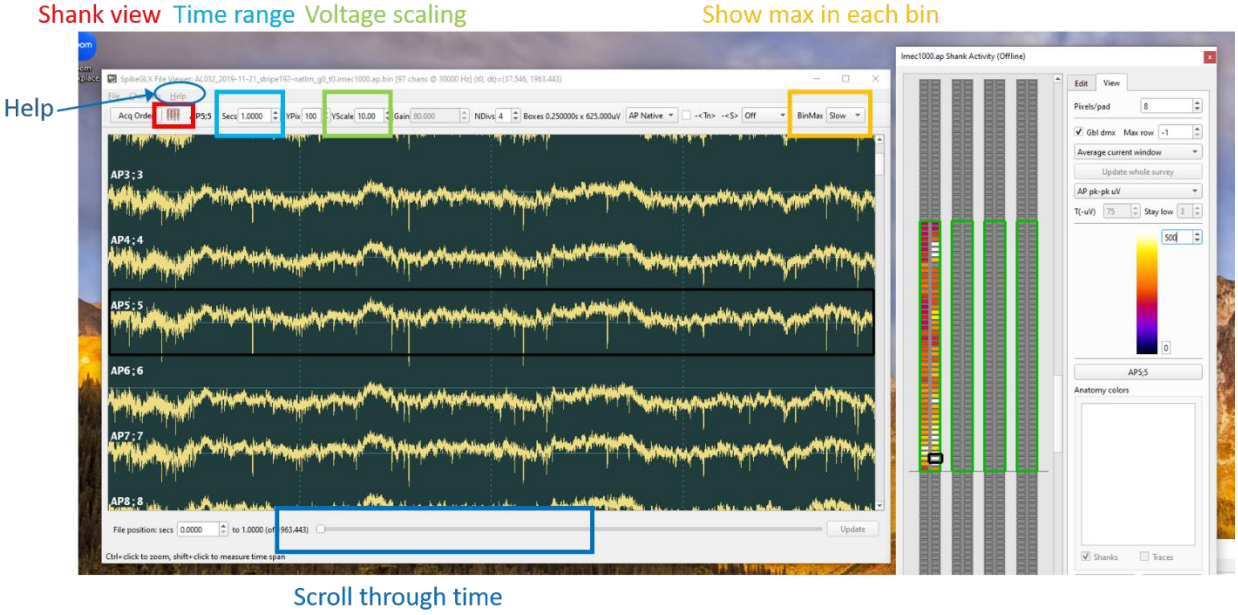
<run_name>_g<gate index>
    <run_name>_g<gate_index>_imec<probe index 0>
    <run_name>_g<gate_index>_imec<probe index 1>
    ....

Each probe folder contains the binary raw data and a metadata file with all the required information to interpret the data: what sites were recorded, how many channels were saved, etc.

To view the data, run the SpikeGLX_NSIM.exe (in the folder: Release_v20251218-api4\SpikeGLX). Select: File->Open File Viewer and navigate into the probe folder, and select the binary. Here's what that looks like in Windows:



The screenshot shows some useful scale settings. You can scroll in time with the bottom scroll bar, and see a summary of the activity by clicking on the shank view button. Click on Help for more options! After running the pipeline, the preprocessed data can be opened in the same viewer—a great way to confirm the preprocessing is having the intended effect.



Run the pipeline on the test data set

We'll be running a simple pipeline to preprocess and sort this recording. It demonstrates how to call a couple of different tools (CatGT for preprocessing, SpikeInterface for sorting, calculation of some extra quality metrics).

Create folder for the sorting output, e.g.:

D:\course_data\output

Open VS code:

View->Explorer to show files in the left hand side bar
File->Open folder to navigate to colorado-neuroopixesl-2026/tutorials/pipelines folder
Double click to open catgt_spikeinterface_pipeline.py
Help->Show All Commands
In the command input, type in 'Python Select Interpreter' and select the npix-test environment.

To run the code:

Set CatGTPath to the path to the CatGT exe (line 33).

In the main() routine, set paths and other information about the data you want to sort e.g.

```
raw_data_parent = r'D:\course_data\AL032\AL032'   # The directory that contains the run folder
run_name = 'AL032_2019-11-21_stripe192-natIm'     # excludes the gate and trigger
gate_str='0'
probe_ind=1000
output_parent = r'D:\course_data\output'
```

The gate string is the part of the run folder name after the 'g', e.g. AL032_2019-11-21_stripe192-natIm_g0 is '0'. The probe index is included in the name of the probe folder and the binary, with the prefix 'imec'. 'imec1000' in the binary name indicates that the probe index is 1000. (Note: This was not a 1000 probe experiment—the index was chosen when extracting the data from shank 0 for this example.)

From here, you can simply run the code. (Run -> Start Debugging, the run button).

Most errors will be due to problems with the folder naming. In particular, if CatGT fails, check the CatGT.log to see if it seems to be looking for the wrong folder.

Steps during the run:

CatGT runs preprocessing. This doesn't report anything to the command line, but the VS code status bar will be blue, indicating its active. Depending on your hard drive, this will take 5-6 miniues.

Spike interface loads the preprocessed recording.

KS4, called from SpikeInterface – its output is streamed to the terminal, so you'll be able to see progress. Run times will be variable, but expect ~10 minutes. If KS4 is running very slowly, check the command line output for GPU memory usage in the 'Resource Usage' blocks. If it is 0, then KS4 is running on the CPU, and that will need to be debugged.

SpikeInterface builds Analyzer object and calculates metrics.

Pipeline code calculates a few more metrics.

Open the output in phy

Open a new conda window, and activate the phy2 environment, then cd to the directory of sorter output:

```
conda activate phy2-test
d:
cd course_data\output\AL032_2019-11-21_stripe192-natIm_g0\kilosort4_output\sorter_output
```

Inside this folder there's a family of output files describing the sort output. A good overview of what's in these files (from Nick Steinmetz) is available here, along with advice about how to use phy on real data: https://phy.readthedocs.io/en/latest/sorting_user_guide/. The folder also contains the params.py file that specifies data details for phy.

We'll talk much more about what's displayed in the course, the main goal right now is to confirm the phy installation. Open the GUI with:

```
phy2 template-gui params.py
```

These warnings in the command line are all normal:

```
(phy2_new) D:\course_data\AL032_output\AL032_2019-11-21_stripe192-natIm_g0\kilosort4_output\sorter_output>phy template-gui params.py
17:37:56.956 [W] model:1416          File D:\course_data\output\AL032_2019-11-21_stripe192-natIm_g0\kilosort4_output\sorter_output\recording.dat does not exist.
17:37:57.263 [W] model:668           Skipping spike waveforms that do not exist, they will be extracted on the fly from the raw data as needed.
17:37:57.267 [W] traces:461          File D:\course_data\output\AL032_2019-11-21_stripe192-natIm_g0\kilosort4_output\sorter_output\recording.dat does not exist.
17:37:57.270 [W] traces:461          File D:\course_data\output\AL032_2019-11-21_stripe192-natIm_g0\kilosort4_output\sorter_output\recording.dat does not exist.
```

The interface will come up with no plots, click on a unit in the ClusterView (top left) to see attributes of that cluster.

The warnings occur because the params.py file is specifying the SpikeInterface internally generated recording.dat file as the raw data. For these data, the preprocessed output from CatGT is a good source for the raw waveforms and amplitudes. To use that binary, edit three fields in params.py:

- n_channels_dat needs to reflect the original data, which includes the sync/status word in addition to the 96 data channels. In general, these details can be found in the metadata file.
- hp_filtered needs to be set to True.
- dat_path needs to point to the CatGT output. Note the use of forward slashes, another way to avoid python interpreting backslashes in the path.

```
n_channels_dat = 97
offset = 0
sample_rate = 30000.0
dtype = 'int16'
hp_filtered = True

dat_path = ['D:/course_data/output/AL032_2019-11-21_stripe192-natIm_g0/AL032_2019-11-21_stripe192-natIm_g0_imec1000/AL032_2019-11-21_stripe192-natIm_g0_tcat.imec1000.ap.bin]
```

Reopening the data with this params.py file will allow display of waveforms from the data, which is useful for assessing jitter and signal to noise.